

CodeKids — CI/CD Pipeline

Volumen VI: Automatización y Despliegue Continuo con Git e Integración de Webhooks

MÓDULO TÉCNICO

Automatización GitOps & Webhooks

DESARROLLADORES CLAVE

Alan / Aquino / Francisco / Alonso

FASE DE DESPLIEGUE

Orquestación e Integración de Código

FECHA DE SISTEMA

Mayo 2026

1. El Flujo de Trabajo GitOps en Render

El ciclo de vida de actualización de código de la plataforma CodeKids se administra bajo el paradigma **GitOps**, vinculando el repositorio privado del equipo directo con el proveedor Cloud Render. Esto significa que la rama principal (`main`) actúa como la única fuente de verdad del estado del software en producción.

2. Pasos Secuenciales para la Vinculación del Repositorio

Para subir el código a producción de manera transparente, el equipo configuró el pipeline de la siguiente manera:

1. **Autenticación OAuth:** Se vinculó la cuenta del repositorio organizacional de GitHub con la consola web de Render, otorgando permisos específicos de lectura sobre la carpeta unificada `CodeKids - Clean/`.
2. **Detección del Manifiesto:** Al configurar el nuevo servicio dentro del panel web de Render, se seleccionó la opción de importar un "Blueprint". Render leyó el archivo `render.yaml` de la raíz e inicializó el entorno de compilación de forma autónoma.
3. **Configuración de Webhooks Nativos:** GitHub inyecta un webhook en el repositorio. Cada vez que los desarrolladores (Alan, Aquino, Francisco o Alonso) realizan un `git push` exitoso a la rama principal, se genera un evento push codificado que notifica a los servidores de Render de forma inmediata.

3. Proceso Interno de Construcción (Build Pipeline)

Cuando Render recibe la notificación del webhook, arranca una secuencia automatizada aislada dentro de sus granjas de servidores en Linux:

Etapas del Pipeline	Acción Técnica Ejecutada	Resultado en CodeKids
Clonación y Checkout	Descarga el delta del último commit confirmado en Git de forma segura.	Garantiza que la versión desplegada corresponda exactamente al último cambio verificado.
Instalación de Dependencias	Analiza el archivo <code>package.json</code> y ejecuta <code>npm install</code> en caché.	Descarga librerías del lado de desarrollo como los scripts purificadores de Tailwind CSS.
Compilación de Activos	Ejecuta el script de construcción configurado (compilación del CSS, validación de scripts).	Genera los archivos optimizados listos para consumo infantil dentro de la carpeta productiva.
Health Checking	Monitorea la integridad del build interno asegurando la lectura del <code>index.html</code> .	Evita caídas del servicio; si el build falla, el despliegue se cancela sin afectar a los usuarios activos.
Atomic Swapping	Intercambia los punteros de red del tráfico de producción hacia los nuevos archivos compilados.	Permite actualizaciones del sistema en caliente con cero tiempo de inactividad (Zero-Downtime Deployment) .

Previene la Saturación de Compilación

Debido a que CodeKids utiliza una integración con el motor Ollama en entornos de desarrollo a través de Firebase Functions, se configuró la bandera `autoDeploy: true` únicamente para ramas estables de producción. Para ramas de desarrollo del equipo, los despliegues automáticos se encuentran inhabilitados para optimizar el uso de minutos de construcción cloud.